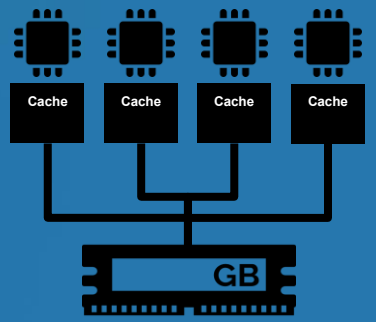
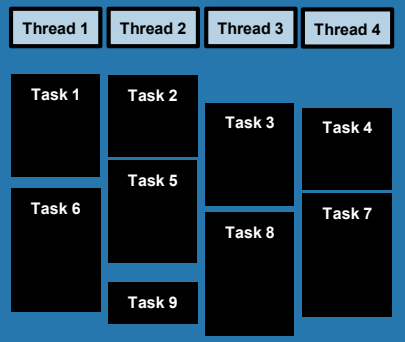
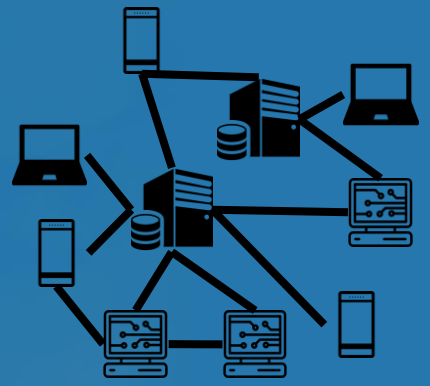
Parameterized Verification 2024

Lecture 2







Organization: Exercises and Discussion Slots

- First Exercise Sheet: next Monday
- Discussion Slots: 20min slots in person during following times:
  - Friday 9-10 (before lecture)
  - Friday 12-14 (after lecture)
  - If neither works for your group, let us know
- Distribution of slots:
  - upload file that contains names of your group members (2-3 students)
  - pick possible tutorial times in CMS
  - (sufficient if one in the group does this)

The Model-Checking Problem

Let

- $\mathcal{P}$ be a set of processes (e.g., all finite-state processes $P = (Q, Q_0, \Sigma, \delta, \lambda)$)
- $card$ be a synchronization constraint (e.g., $card = \{1\}$ for pairwise rendezvous)
- $\mathcal{F}$ be a set of specifications (e.g., reachability of subsets of Q^n)

Then the model-checking problem for $\mathcal{P}, card$ and $\mathcal{F}$ is defined by

Input: $P \in \mathcal{P}, n \in \mathbb{N}$ and $\varphi \in \mathcal{F}$

Output: "Yes" if $P^n_{card} \vDash \varphi$, "No" otherwise.

Reachability Specifications in Parameterized Systems

In systems P^n of fixed size n, we are interested in reachability of subsets of Q^n

If we are interested in parameterized correctness arguments, this does not work:
any subset of Q^n is only reachable in a system with exactly n processes

$\Rightarrow$ need a more general type of specifications

For $m > n$, we say that a state $s' \in Q^m$ contains a state $s \in Q^n$ if there is an injective mapping $f: \{1, \dots, n\} \to \{1, \dots, m\}$ such that $s(i) = s'\big(f(i)\big)$

Examples:

(q) is contained in $q, (q, q), (q, q'), (q', q), (q_1, q_2, \dots, q_n, q)$

(q_1, q_2) is contained in $(q, q_1, q_2), (q_1, q, q_2)$ and in (q_2, q_2, q_1)

(q, q) is not contained in (q, q') or (q_1, q_2, q)

Reachability Specifications in Parameterized Systems (II)

For $R \subseteq Q^n$, let $\uparrow R = \bigcup_{m \geq n} \{s \in Q^m \mid s \text{ contains } s' \text{ for some } s' \in R\}$.

We call $\uparrow R$ the (upward) closure of R.

Examples:

$\uparrow \{(q)\} = \{(q)\} \cup \{(q, q'), (q', q) \mid q' \in Q\} \cup \{(q, q', q''), (q', q, q''), (q', q'', q) \mid q', q'' \in Q\} \cup \cdots$

$\uparrow \{(q_1, q_2)\} = \{(q_1, q_2), (q_2, q_1)\} \cup \{(q_1, q_2, q), (q_1, q, q_2), \ldots \mid q \in Q\} \cup \cdots$

For parameterized correctness (in completely symmetric systems),
we are interested in reachability of $\uparrow R$, for some $R \subseteq Q^n$.

$\uparrow R$ contains states of arbitrary size,
even though $R \subseteq Q^n$ for fixed n.

Examples: Reachability Specifications in Parameterized Systems

Reachability of a local error state:

if we want to check whether some local state q_E is reachable in a system of any size,

we check reachability of states from $\uparrow R$ for $R = \{q_E\}$

Mutual exclusion:

if we want to check whether two processes can reach critical state q_C at the same time (in a system of any size),

we check reachability of states from $\uparrow R$ for $R = \{(q_C, q_C)\}$

Deadlock Detection in Parameterized Systems

In contrast to specifications mentioned before, reachability of a deadlock state is inherently a global property:

can we reach a global state where for all processes no transition is possible?

This cannot be expressed as a set $\uparrow R$ for $R \subseteq Q^n$, and is treated as a special case (if necessary).

The Parameterized Model-Checking Problem (PMCP)

Let

- $\mathcal{P}$ be a set of processes (e.g., all finite-state processes $P = (Q, Q_0, \Sigma, \delta, \lambda)$)
- $card$ be a synchronization constraint (e.g., $card = \{1\}$ for pairwise rendezvous)
- $\mathcal{F}$ be a set of specifications (e.g., reachability of states from $\uparrow R$ for some $\mathrm{R} \subseteq Q^m$)

Then the parameterized model-checking problem (PMCP) for $\mathcal{P}, card$ and $\mathcal{F}$ is defined by

Input: $P \in \mathcal{P}$ and $\varphi \in \mathcal{F}$

Output: "Yes" if $\forall n \in \mathbb{N}: (P^n_{card} \vDash \varphi)$, "No" otherwise.

How can such problems be solved?

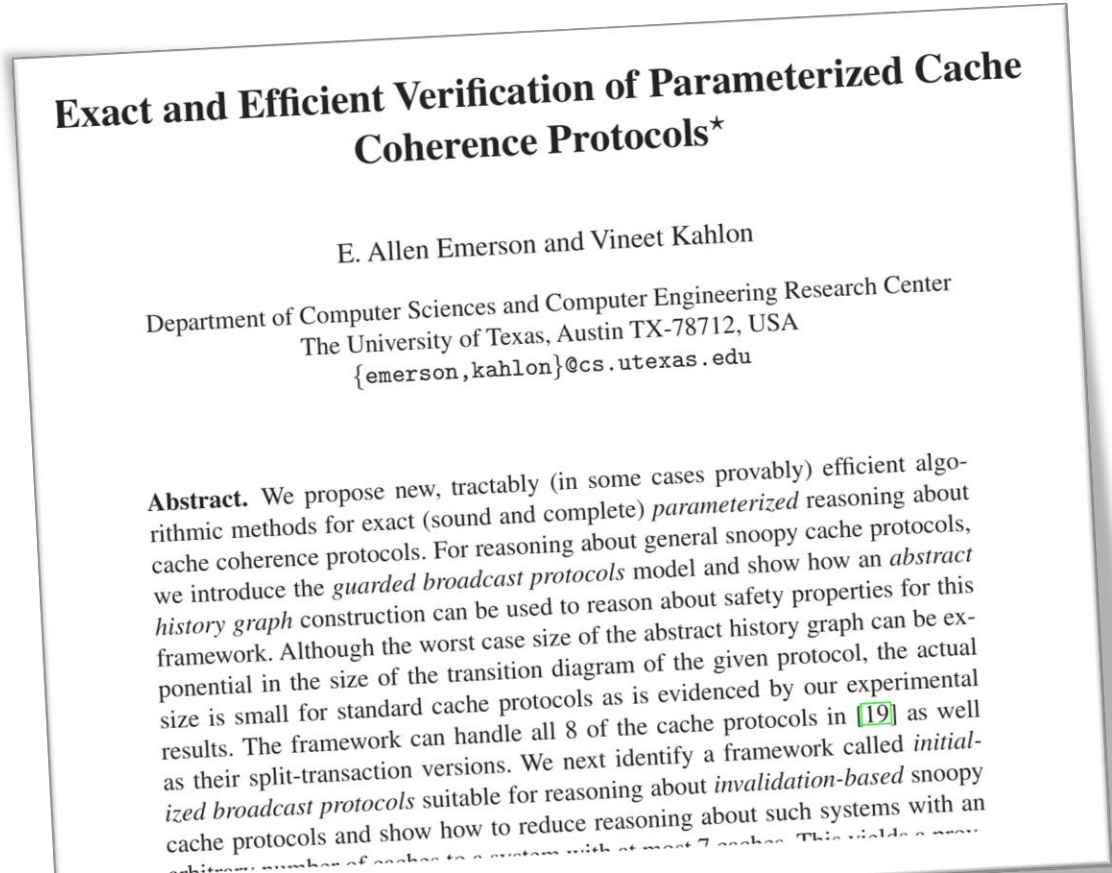
Intuition about Parameterized Correctness

If $\forall n \in \mathbb{N}: (P_{card}^{n} \vDash \varphi)$ does not hold, there usually is a "small" $m \in \mathbb{N}$ such that $P_{card}^{m} \nvDash \varphi$

This intuition holds in many cases.

Example:

To verify safety properties in a certain model of cache coherence protocols, it is always enough to consider at most 7 processes.



Exact and Efficient Verification of Parameterized Cache Coherence Protocols*

E. Allen Emerson and Vineet Kahlon

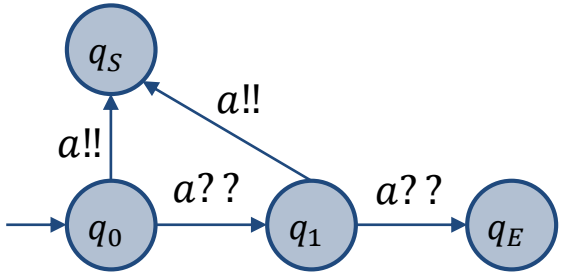
Department of Computer Sciences and Computer Engineering Research Center
The University of Texas, Austin TX-78712, USA
{emerson,kahlon}@cs.utexas.edu

Abstract. We propose new, tractably (in some cases provably) efficient algorithmic methods for exact (sound and complete) *parameterized* reasoning about cache coherence protocols. For reasoning about general snoopy cache protocols, we introduce the *guarded broadcast protocols* model and show how an *abstract history graph* construction can be used to reason about safety properties for this framework. Although the worst case size of the abstract history graph can be exponential in the size of the transition diagram of the given protocol, the actual size is small for standard cache protocols as is evidenced by our experimental results. The framework can handle all 8 of the cache protocols in [19] as well as their split-transaction versions. We next identify a framework called *initialized broadcast protocols* suitable for reasoning about *invalidation-based* snoopy cache protocols and show how to reduce reasoning about such systems with an

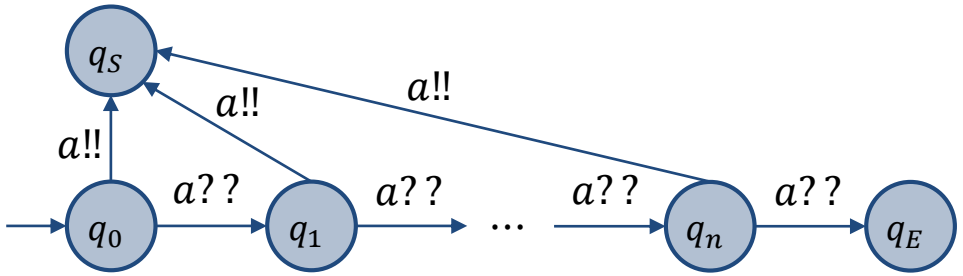
But it does not always hold!

Example: PMCP in Broadcast Protocol

Reachability of $\uparrow \{q_E\}$ in broadcast protocol, i.e., $card = \mathbb{N}_0$:



Processes needed to reach q_E: 3



Processes needed to reach q_E: $n + 2$

Cutoffs

The notion of cutoffs formalizes the intuition that in some cases a small number of processes is enough to prove parameterized correctness.

A number $c \in \mathbb{N}$ is a cutoff for the PMCP for $\mathcal{P}, card$ and $\mathcal{F}$ if for all $P \in \mathcal{P}, \varphi \in \mathcal{F}$:

$$\forall n \geq c: (P^n_{card} \vDash \varphi \;\Leftrightarrow\; P^c_{card} \vDash \varphi)$$

⇨ cutoffs reduce the PMCP to a finite number of model-checking problems

⇨ finding a cutoff is one approach for solving a given PMCP

Symmetry and Abstractions

A second way to solve a PMCP is by using symmetry and abstractions:

- we simplify the system such that we can handle it efficiently
- while preserving the properties that we are interested in (but possibly not others)

Which abstractions can be used depends on the systems and properties we are interested in.

We will cover a few of them in this course.

Decidability Results: Broadcast Protocols

Broadcast Protocols: Motivation

"Real" broadcast communication is widely used in practice:
cellular networks, wifi, IoT/autonomous vehicles, etc.

Other cases can be viewed for our purposes as broadcast communication,
e.g., communication over a shared bus that multiple components can listen to

Broadcast communication is one of the most powerful and therefore most important in theory:
it can simulate most of the other synchronization primitives we will see later

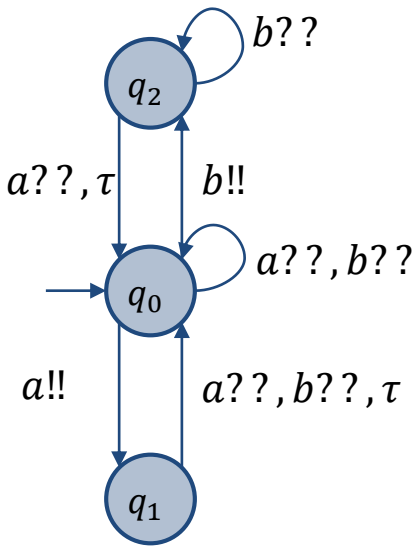
Example: Synchronization by Broadcast

$card = \mathbb{N}_0$

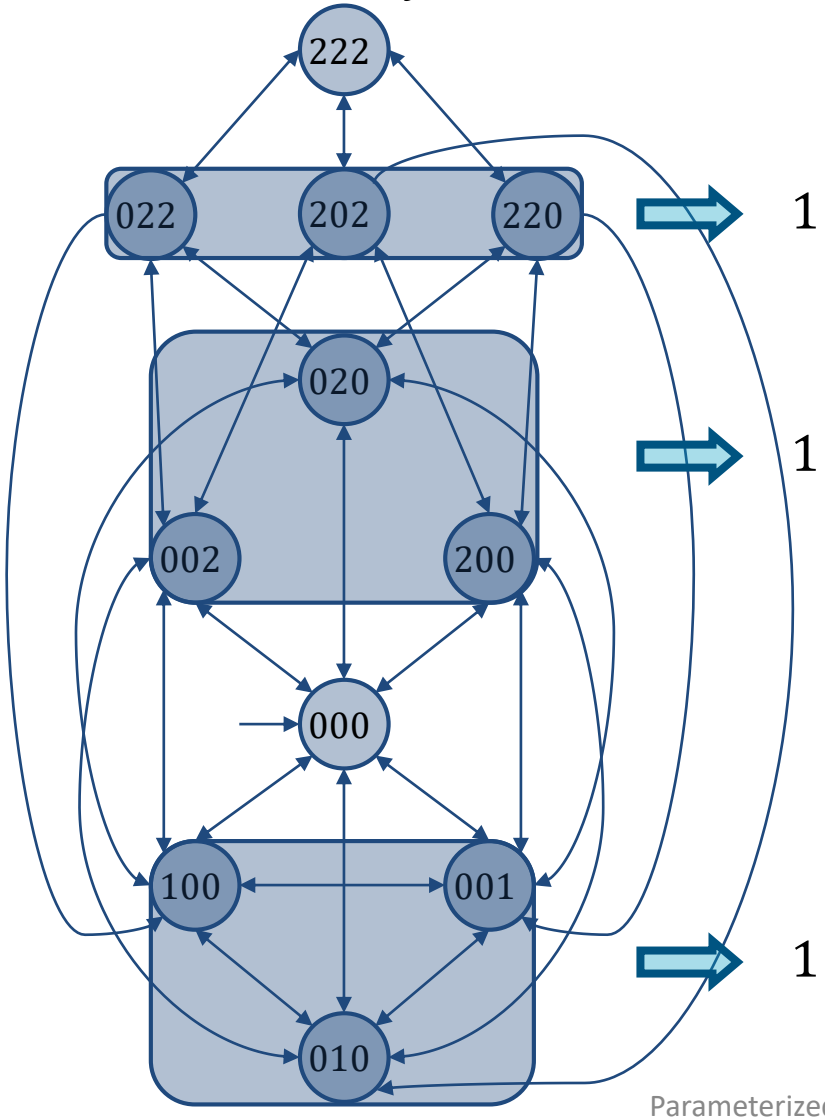
$a!! = out_a$

$a?? = in_a$

Process M



Composition M^3
for $\Sigma_{int} = \{\tau\}$, $\Sigma_{sync} = \{a, b\}$



1 process in q_0, 2 in q_2

1 process in q_2, 2 in q_0

1 process in q_1, 2 in q_0

Counter System of a Broadcast Protocol

Idea:

Instead of keeping track of exactly which process is in which local state,
just keep track of how many processes are in each local state

Given a system P^n with state space $S = \{q_1, \dots, q_m\}^n$, the abstraction of a state s is defined as

$abs(s) = (i_1, \dots, i_m)$ with $i_j = \left|\{p \in \{1, \dots, n\} \mid s(p) = q_j\}\right|$. ($i_j$ "counts" how many processes are in q_j)

Examples (in system with $Q = \{q_0, q_1, q_2\}$):

$abs(q_0, q_0, q_0) = (3,0,0)$

$abs(q_0, q_2, q_0) = (2,0,1)$

$abs(q_0, q_0, q_0, q_0, q_0, q_0, q_0, q_0, q_0) = (9,0,0)$

$abs(q_0, q_1, q_1, q_2, q_0, q_2, q_1, q_2, q_0) = (3,3,3)$

Counter System of a Broadcast Protocol (II)

Let $P = (Q, Q_0, \Sigma, \delta, \lambda)$ be a process based on actions Σ_{int} and Σ_{sync}, and let $card = \mathbb{N}_0$, i.e., we consider broadcast synchronization.

The counter system for P^n is the transition system $(C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ with

- $C = \{0, \dots, n\}^{|Q|}$, i.e., each abstract state c $\in C$ can also be seen as a function c: $Q \to \{0, \dots, n\}$
- $C_0 = \{c \in C \mid c(q) = 0 \text{ if } q \notin Q_0\}$
- Δ_{abs} contains a transition $abs(s) \xrightarrow{a} abs(s')$ iff $s \xrightarrow{a} s'$ is a transition in P^n

(these abstract systems are usually considered without state labels)

Assumptions on Broadcast Protocols

An LTS $(Q, Q_0, \Sigma, \delta, \lambda)$ is deterministic on $\Sigma' \subseteq \Sigma$ if for each state $q \in Q$ and each action $a \in \Sigma'$, there is at most one transition $q \xrightarrow{a} q'$ in δ.

To simplify presentation, in the following we assume that

- process P is deterministic on broadcast receives, i.e., on actions $a??$ for $a \in \Sigma_{sync}$
- each broadcast send $a!!$ appears on exactly one transition in δ
- every state has a transition on every broadcast receive $a??$

(a system can be transformed into this form by renaming duplicate broadcast send actions, and introducing additional broadcast receive actions for these new send actions; similarly for nondeterministic broadcast receives; and by adding self-loops on broadcast receives if no other receive transition is possible)

Expressing Broadcast Transitions as Operations on Vectors

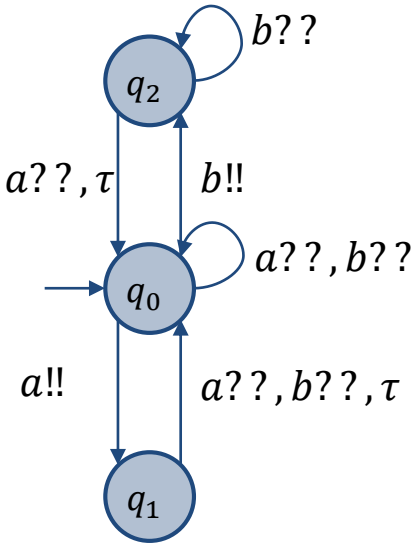
Under these assumptions, compute the effect of an arbitrary broadcast action a as follows.

Let u_i be the unit vector with $u_i(q_i) = 1$ and $u_i(q) = 0$ for $q \neq q_i$.

If $q_i \xrightarrow{a!!} q_j$ is the unique local transition with action $a!!$
and c is an abstract state with $c(q_i) \geq 1$, then: (otherwise the transition cannot be taken from abstract state c)

1. let $c_1 = c - u_i$ — broadcast sender leaves q_i
2. for each q_k, let $c_2(q_k) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_k\}} c_1(q_l)$ — all other processes receive the broadcast
3. let $c' = c_2 + u_j$ — broadcast sender moves to q_j

Example: Broadcast Transitions as Operations on Vectors



$q_i \xrightarrow{a!!} q_j$ is local transition with action $a!!$
and c abstract state with $c(q_i) \geq 1$:

1. let $c_1 = c - u_i$
2. for each q_k, let $c_2(q_k) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_k\}} c_1(q_l)$
3. let $c' = c_2 + u_j$

Compute effect of action a on $c = (4,0,2)$:

local sending transition is $q_0 \xrightarrow{a!!} q_1$

$c_1 = c - u_0 = (3,0,2)$

$c_2(q_0) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_0\}} c_1(q_l) = c_1(q_0) + c_1(q_1) + c_1(q_2) = 5$

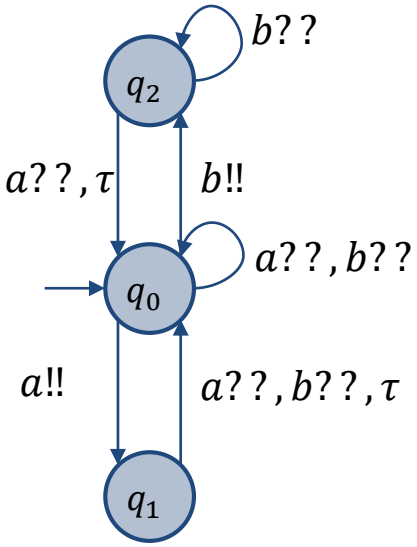
$c_2(q_1) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_1\}} c_1(q_l) = 0$

$c_2(q_2) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_2\}} c_1(q_l) = 0$

$c_2 = (5,0,0)$

$c' = c_2 + u_1 = (5,1,0)$

Example: Broadcast Transitions as Operations on Vectors (II)



$q_i \xrightarrow{a!!} q_j$ is local transition with action $a!!$
and c abstract state with $c(q_i) \geq 1$:

1. let $c_1 = c - u_i$
2. for each q_k, let $c_2(q_k) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_k\}} c_1(q_l)$
3. let $c' = c_2 + u_j$

Compute effect of action b on $c = (4,0,2)$:

local sending transition is $q_0 \xrightarrow{b!!} q_2$

$c_1 = c - u_0 = (3,0,2)$

$c_2(q_0) = \sum_{\{q_l \mid q_l \xrightarrow{b??} q_0\}} c_1(q_l) = c_1(q_0) + c_1(q_1) = 3$

$c_2(q_1) = \sum_{\{q_l \mid q_l \xrightarrow{b??} q_1\}} c_1(q_l) = 0$

$c_2(q_2) = \sum_{\{q_l \mid q_l \xrightarrow{b??} q_2\}} c_1(q_l) = c_1(q_2) = 2$

$c_2 = (3,0,2)$

$c' = c_2 + u_2 = (3,0,3)$

Expressing Broadcast Receive as a Matrix Multiplication

Operations in point 1 and 3 of the computation are simple additions of vectors.

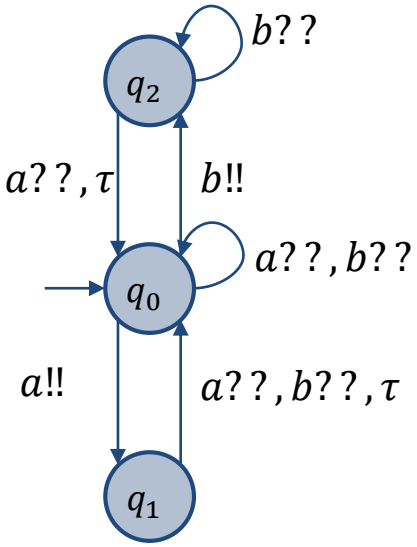
The operation in point 2 can be represented as a matrix multiplication:

c_2 with $c_2(q_k) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_k\}} c_1(q_l)$ is the same as $c_2 = M_a \cdot c_1$ for the matrix

$$M_a = \begin{pmatrix} m_{0,0} & \cdots & m_{0,|Q|-1} \\ \vdots & \ddots & \vdots \\ m_{|Q|-1,0} & \cdots & m_{|Q|-1,|Q|-1} \end{pmatrix} \text{ with } m_{k,l} = \begin{cases} 1 \text{ if } q_l \xrightarrow{a??} q_k \\ 0 \text{ otherwise} \end{cases}$$

Note: each column of M_a is a unit vector, and the ith column determines where processes from state q_i go upon a broadcast receive

Example: Broadcast Transitions in Matrix Form



$q_i \xrightarrow{a!!} q_j$ is local transition with action $a!!$
and c abstract state with $c(q_i) \geq 1$:

1. let $c_1 = c - u_i$
2. for each q_k, let $c_2(q_k) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_k\}} c_1(q_l)$
3. let $c' = c_2 + u_j$

Compute effect of action a on $c = \begin{pmatrix} 4 \\ 0 \\ 2 \end{pmatrix}$:

local sending transition is $q_0 \xrightarrow{a!!} q_1$

$$M_a = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

processes from q_0 go to q_0

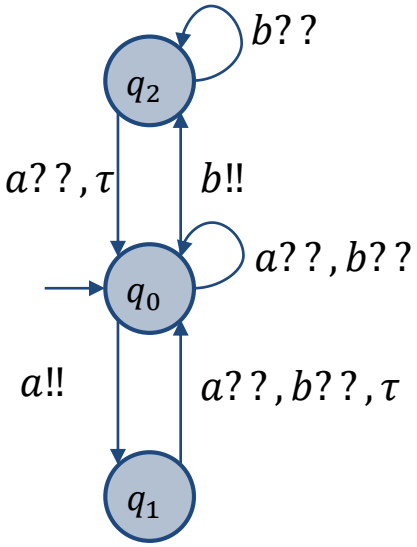
processes from q_1 and q_2 go to q_0

$$c_1 = c - u_0 = \begin{pmatrix} 3 \\ 0 \\ 2 \end{pmatrix}$$

$$c_2 = M_a \cdot c_1 = \begin{pmatrix} 5 \\ 0 \\ 0 \end{pmatrix}$$

$$c' = c_2 + u_1 = \begin{pmatrix} 5 \\ 1 \\ 0 \end{pmatrix}$$

Example: Broadcast Transitions in Matrix Form (II)



$q_i \xrightarrow{a!!} q_j$ is local transition with action $a!!$
and c abstract state with $c(q_i) \geq 1$:

1. let $c_1 = c - u_i$
2. for each q_k, let $c_2(q_k) = \sum_{\{q_l \mid q_l \xrightarrow{a??} q_k\}} c_1(q_l)$
3. let $c' = c_2 + u_j$

Compute effect of action b on $c = \begin{pmatrix} 4 \\ 0 \\ 2 \end{pmatrix}$:

local sending transition is $q_0 \xrightarrow{b!!} q_2$

$$M_b = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

processes from q_0 and q_1 go to q_0

processes from q_2 go to q_0

$$c_1 = c - u_0 = \begin{pmatrix} 3 \\ 0 \\ 2 \end{pmatrix}$$

$$c_2 = M_b \cdot c_1 = \begin{pmatrix} 3 \\ 0 \\ 2 \end{pmatrix}$$

$$c' = c_2 + u_2 = \begin{pmatrix} 3 \\ 0 \\ 3 \end{pmatrix}$$

Counter Systems for Model-Checking Large Systems

Counter systems (with or without matrix form) allow us to model-check large systems more efficiently:

we can modify the model-checking algorithm (on the right) to use abstract states to represent $Reached$, and the approach shown before to compute $\delta(Reached)$

However, until now it does not enable parameterized verification:

- either we start with a fixed-size system, then we only verify that size
- or we start with initial states of all sizes (which is infinite even in counter system), then we can't compute $\delta(Reached)$

To check reachability of R in M, we can for example use the following algorithm, where $\delta(Reached)$ is the set of all states that are in $Reached$, or are reachable in one step of δ :

1. Let $Reached = Q_0$
2. Let $Reached' = \delta(Reached)$
3. If $R \cap Reached' \neq \emptyset$ then return "reachable"
4. If $Reached' \neq Reached$ then {let $Reached = Reached'$; goto 2}
5. Return "unreachable"

How can we extend it to support parameterized verification?

Parameterized Systems

Let $P = (Q, Q_0, \Sigma, \delta, \lambda)$ be a process, $card$ a synchronization constraint.

The parameterized system $sys(P, card)$ is the sequence

$$\left(P^1_{card}, P^2_{card}, \dots\right).$$

We call

- P^n_{card} an instance (if size n) of the parameterized system
- $\bigcup_{n \in \mathbb{N}} Q^n$ the states of the parameterized system,
- $\bigcup_{n \in \mathbb{N}} Q^n_0$ the initial states of the parameterized system.

Counter System of a Parameterized System

Let $P = (Q, Q_0, \Sigma, \delta, \lambda)$ be a process based on actions Σ_{int} and Σ_{sync}, and let $card = \mathbb{N}_0$, i.e., we consider broadcast synchronization.

The counter system for $sys(P, card)$ is the transition system $(C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ with

- $C = \mathbb{N}^{|Q|}$, i.e., each abstract state $c \in C$ can also be seen as a function $c: Q \to \mathbb{N}$
- $C_0 = \{c \in C \mid c(q) = 0 \text{ if } q \notin Q_0\}$
- Δ_{abs} contains a transition $abs(s) \xrightarrow{a} abs(s')$ iff $s \xrightarrow{a} s'$ is a transition in P^n for some $n \in \mathbb{N}$

Essentially: starts in initial state of arbitrary size, then takes transitions in instance of that size.

(unless mentioned otherwise, we always consider parameterized systems where number of processes does not change during a run)

How can we efficiently compute with an infinite number of states?

Well-Quasi-Orders

Let S be an infinite set, $\lesssim \subseteq S \times S$ a binary relation on S.
Then $\lesssim$ is a well-quasi-order (wqo) on S if the following hold:

1. Reflexive: $\forall s \in S: s \lesssim s$
2. Transitive: $\forall s, t, u \in S: s \lesssim t \wedge t \lesssim u \rightarrow s \lesssim u$
3. Any infinite sequence $s_0, s_1, s_2, \ldots \in S^\omega$ contains a pair $s_i \lesssim s_j$ with $i < j$.
   I.e., it cannot be infinitely decreasing or contain an infinite antichain (sequence of incomparable elements).

Examples:

- $\leq$ is a wqo on $\mathbb{N}$
- $\leq$ is not a wqo on $\mathbb{Z}$ (infinite sequence $0, -1, -2, -3, \ldots$)
- divisibility is not a wqo on $\mathbb{N}$ (infinite sequence $2,3,5,7,11, \ldots$ of prime numbers)
- subsequence ordering is a wqo over finite strings

Properties of Well-Quasi-Orders

Let S be an infinite set, $\lesssim \subseteq S \times S$ a wqo on S.

Then:

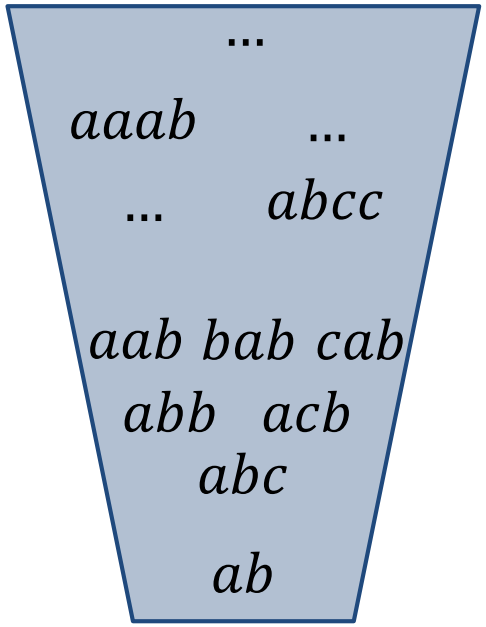
- the upward closure of a set $S' \subseteq S$, denoted $\uparrow S'$, is the set $\{s \in \mathrm{S} \mid \exists s' \in S' : s' \lesssim s\}$
  (**Note**: this is a generalization of the upward closure $\uparrow R$ on states in parameterized systems we have seen before; we will later show when they coincide)
- we call a set $S' \subseteq S$ upward-closed if $\uparrow S' = S'$
- we call $B \subseteq S$ a basis of an upward-closed set S' if $\uparrow B = S'$
- if S' is upward-closed, then it has a finite minimal basis,
  i.e., a basis consisting only of elements $s \in S$ such that $\nexists s' \in S : s' \preccurlyeq s$.
  (if it was not finite, then the set of minimal elements would constitute an infinite antichain)

Example: Properties of Well-Quasi-Orders

Consider subsequence ordering over finite strings, with alphabet $\Sigma = \{a, b, c\}$.

If S' is upward-closed, then it has a finite minimal basis,
i.e., a basis consisting only of elements $s \in S$ such that $\nexists s' \in S : s' \preccurlyeq s$.

$\uparrow \{ab\}$:



every basis of $\uparrow \{ab\}$ must contain ab, since it is minimal wrt. the subsequence ordering

the set of minimal elements in any upward closed set must be finite – otherwise it would constitute an infinite antichain, which is not possible with a wqo

Specifications in Parameterized Systems: Reachability of an Upward-Closed Set

Consider the reachability of $\uparrow R$ with $R \subseteq Q^n$ in a parameterized system $sys(P, card)$ with $P = (Q, Q_0, \Sigma, \delta, \lambda)$ (as seen on Slide 11 of this lecture)

R can be interpreted as a (finite) basis of $\uparrow R$,
where $\uparrow R \subseteq \mathbb{N}^{|Q|}$ is an infinite set of abstract states.

That is:
to handle infinite sets of abstract states (from instances of the parameterized system with arbitrary size), we want a wqo $\lesssim \subseteq \mathbb{N}^{|Q|} \times \mathbb{N}^{|Q|}$ that relates abstract states of systems with different size

Examples: $(2,0,0) \lesssim (3,0,0) \lesssim (3,0,1)$, etc.

Well-Quasi-Order on States of a Counter System

Theorem:
Let $S = \mathbb{N}^k$ for some $k \in \mathbb{N}$. For $s = (i_1, \dots, i_k) \in S$, let $s(j)$ denote i_j, the jth component of s. Then the relation $\preccurlyeq \subseteq S \times S$ defined by

$$s \preccurlyeq s' \iff \forall j \in \{1, \dots, k\}: s(j) \leq s'(j)$$

is a well-quasi-order on S. It is also called component-wise ordering.

Proof sketch: Clearly, $\preccurlyeq$ is reflexive and transitive.

To see that the third property of a wqo holds, assume that we have an infinite sequence $s_0, s_1, s_2, \dots$ that does not contain a pair $s_i \preccurlyeq s_j$ with $i < j$. Such a sequence must contain either a subsequence

- with infinitely many strictly decreasing elements, i.e., $s_i \succcurlyeq s_j \wedge s_i \neq s_j$ for $i < j$, or
- with infinitely many incomparable elements, i.e., $\neg(s_i \preccurlyeq s_j) \wedge \neg(s_i \succcurlyeq s_j)$ for $i < j$

The first is not possible, since for any s_i there are only finitely many elements strictly smaller than s_i.

The second is also not possible: s_i and s_j are incomparable if there are k, l with $s_i(k) < s_j(k)$ and $s_i(l) > s_j(l)$. In particular, an element s_j in a sequence of pairwise incomparable elements must be strictly smaller in at least one component than all preceding elements. For any s_0, this is only possible finitely often.

Review/Lessons Learned in this Lecture

Topics:

- The parameterized model checking problem
- Basic definitions: cutoffs
- Broadcast protocols: definition, checking safety/reachability properties

Techniques:

- Counter systems
- Well-quasi orders